VAS VÁRMEGYEI SZAKKÉPZÉSI CENTRUM

HORVÁTH BOLDIZSÁR

KÖZGAZDASÁGI ÉS INFORMATIKAI

TECHNIKUM

A 5 0613 12 03 számú Szoftverfejlesztő és –tesztelő vizsgaremek

A szoftveralkalmazás dokumentációja

EuroTrip

Készítették:

KOVÁCS BARNABÁS

MUCZA JÁNOS

NAGY DONÁT

SZOMBATHELY

2026

Tartalom

1. Bevezetés ... 3
1.1. A szoftver célja ... 3
1.2. Célközönség ... 3
2. Program általános leírása ... 4
2.1. Követelmények, tervezés és a rendszer jellemzői – megvalósítási terv ... 4
2.2. Adatbázis kapcsolathoz ... 5
2.3. Tervezés, munkamegosztás ... 5
3. Adatbázis ER modell, leképezés ... 6
3.1. Táblák és mezők ... 6
4. REST API végpontok részletes specifikációja ... 9
4.1. Hitelesítés és jogosultságkezelés (Login Controller) ... 9
4.2. Városok kezelése és relációs adatlekérés (CityController) ... 10
4.3. Országok listázása (CountryController) ... 11
4.4. Szálláshelyek kezelése (AccommodationController) ... 11
4.5. Szobafoglalási rendszer (RoomBookingController) ... 11
4.6. Éttermek és asztalfoglalások (RestaurantController, TableReservationController) 12
4.7. Globális konfiguráció és biztonság (Program.cs) ... 13
5. Biztonsági architektúra és hitelesítés ... 13
5.1. Kriptográfiai jelszóvédelem (Argon2id) ... 14
5.2. JWT alapú hitelesítés és token menedzsment ... 14
5.3. Szerepkör alapú jogosultságkezelés ... 15
5.4. A regisztrációs folyamat logikája ... 17
5.5. Hitelesítés és jelszóellenőrzés ... 18
6. Kliensoldali architektúra és felhasználói felület (Frontend) ... 19
6.1. Reaktív komponens alapú felépítés ... 19

6.2. Globális konfiguráció és belépési pont(main.jsx) ... 20
6.3. Dinamikus útvonalválasztás (React Router) ... 20
6.4. Aszinkron állapotkezelés ... 20
6.5. A felhasználói felület részletes komponens-elemzése ... 21
7. Dizájn és CSS ... 24
7.1. Layout: flexbox és grid kapcsolata ... 25
7.2. Dinamikus témakezelés és sötét mód ... 25
7.3. Vizuális megjelenés és kártyás felépítés ... 26
7.4. Animációk és felhasználói visszajelzések ... 27
7.5. Reszponzivitás ... 27
8. Weboldal technikai leírása ... 28
8.1. Komponenseinek technikai leírása ... 28
8.2. Működésének műszaki feltételei ... 30
8.3. Használatának rövid bemutatása ... 30
Regisztráció ... 31
Bejelentkezés ... 32
9. Tesztek ... 33
9.1. Tesztelési környezet ... 33
9.2. Adatbázis implementálása ... 33
9.3. Objektum összehasonlítás ... 33
9.4. Tesztosztályok ... 34
9.5. Részletes CRUD tesztelés ... 35
10. Segédoldalak, információs felületek, linkek: ... 35
11. Továbbfejlesztési lehetőségek ... 36
Ábrajegyzék ... 38

1. Bevezetés

1.1. A szoftver célja

Csapatunk egy EuroTrip nevezetű weboldalt készített, mely segít egy adott utazás megszervezésében, azon belül éttermek és szállások kínálatát tárja elénk. Azért választottuk ezt a témát, mert szerintünk kevés olyan megfelelően szerkesztett, létrehozott weboldal van, amely egy tartalmas, de ugyanakkor átlátható és egyszerűen értelmezhető képet ad arról, hogy Európán belül az adott országokban, mely városokat érdemes meglátogatni. Hol lehet egy jót étkezni, illetve a szállás sem egy utolsó szempont.

Ezért csapatunk elhatározta, hogy készít egy olyan szoftvert/weblapot, amely összegyűjti az egyes országok nevezetesebb városait, éttermeit, szállásait. Például, ha valaki elhatározta, hogy Spanyolországba, azon belül Madridba szeretne elutazni, akkor ez az oldal tökéletes választás számára, hogy egy átláthatóbb képet kapjon a városról. Láthatja Madrid éttermeit, szállásait, így előre tud tervezni és a számára legkedvezőbb, legszimpatikusabb opciót választhatja. Lefoglalhat az oldalon keresztül mind éttermet, mind szállást.

A felhasználók nem csak időt és energiát spórolnak meg, hanem pénzt is. Ugyanis a weboldal látogatása teljesen díjmentes, ennek következtében a felhasználóknak nem kell fölöslegesen pénzt költenie különböző utazócsoportok, oldalak igénybevételéhez. Az összes fontos információt megtalálják egy helyen és maga az utazás alatt is bármikor informálódhatok a weboldalról.

1.2. Célközönség

Az EuroTrip webalkalmazás tervezése során a "User-Centered Design" (Felhasználóközpontú tervezés) elveit követtük. Mivel a szoftver egy integrált turisztikai platform, a célközönségünk elsősorban a B2C (Business-to-Consumer), azaz a végfelhasználói szegmensre fókuszál, kiegészítve a rendszert karbantartó adminisztrátori körrel. A szoftver célcsoportjait a következő felhasználói perszónákra (szerepkörökre) bonthatjuk:

- **Tudatos, egyéni utazók**
    - Ez a platformunk elsődleges célcsoportja. Ide tartoznak azok a digitálisan érett felhasználók, akik elutasítják a hagyományos, kötött utazási irodai csomagokat, és szeretik saját maguk megszervezni az útjukat. Számukra az EuroTrip megadja a maximális szabadságot: nemcsak a szállásokat válogathatják ki egy adott európai nagyvárosban, hanem előre lefoglalhatják a hozzájuk legközelebb eső, kiváló értékelésű

éttermeket is. Számukra a platform transzparenciája és a közvetlen foglalási lehetőség a legnagyobb vonzerő.

- **Költségtudatos turisták**
    - Mivel az EuroTrip platform használata a végfelhasználók számára teljesen díjmentes, ideális választás a költséghatékony utazást tervező diákoknak, fiatal felnőtteknek és családoknak. A felhasználóknak nem kell rejtett közvetítői díjakkal vagy utazási irodai jutalékokkal számolniuk. A beépített szűrőrendszer (például a csillagok alapján történő keresés) lehetővé teszi számukra, hogy a saját pénztárcájuknak megfelelő kompromisszumokat hozzák meg a szállás és az étkezés kiválasztásakor.
- **Időhiányos szakemberek és üzleti utazók**
    - Egy üzleti út megszervezése során a gyorsaság és a megbízhatóság a legfontosabb tényező. Ezt a célközönséget a weboldal letisztult, reszponzív (mobilbarát) felülete, a gyors adatbetöltés, valamint az "egy-kattintásos" foglalási élmény szolgálja ki. Mivel a rendszer egyetlen ökoszisztémában kezeli az éttermi és szállásfoglalásokat, drasztikusan lecsökkenti a logisztikára fordítandó időt: a felhasználónak nem kell 3-4 különböző weboldalt vagy applikációt párhuzamosan használnia.
- **Rendszeradminisztrátorok és üzemeltetők**
    - Bár a platform kifelé a turistákat szolgálja ki, a rendszer működtetéséhez szükség van egy belső felhasználói körre is. A regisztrált felhasználók közül az "Adminisztrátor" jogosultsággal rendelkező személyek felelnek a háttéradatbázis (MySQL) karbantartásáért. Ők azok, akik az API végpontokon keresztül felügyelik az új országok, városok, szállások és éttermek felvitelét, és biztosítják a feltöltött adatok (pl. Cloudinary képek URL-jei, férőhelyek) minőségét és naprakészségét.

2. Program általános leírása

2.1. Követelmények, tervezés és a rendszer jellemzői – megvalósítási terv

Program indítása a frontend mappából npm run dev-vel történik.

React:

• react

• react-dom

Dátumkezeléshez:

• date-fns

Ikonokhoz:

• react-icons Material UI komponensekhez (gomb, tooltip.):

• @mui/material

• @emotion/react

• @emotion/styled

Email küldéshez:

• emailjs/browser

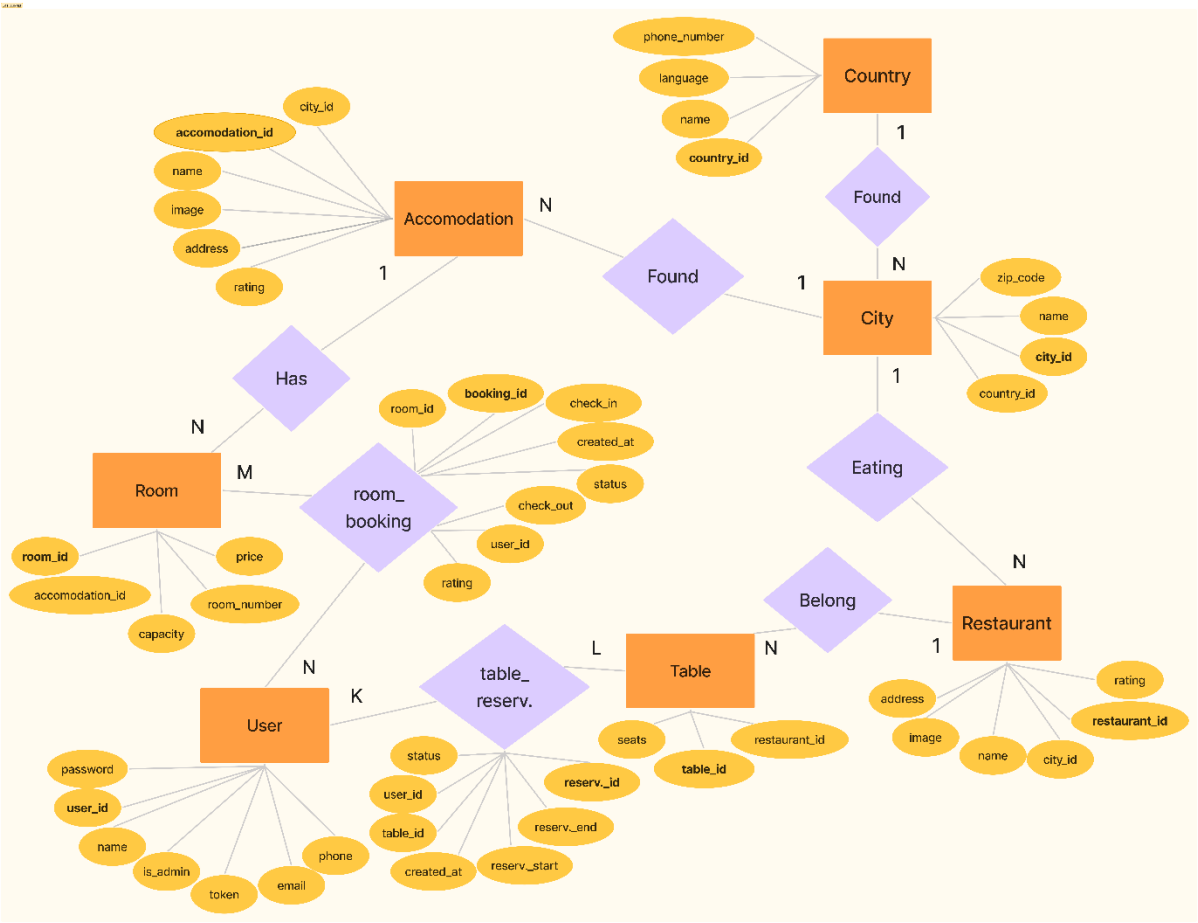
2.2. Adatbázis kapcsolathoz

A XAMPP a MySQL port 3307

2.3. Tervezés, munkamegosztás

Feladat:	**Készítette:**
Adatbázis	Nagy Donát
Entity realitionship diagram	Nagy Donát
ASP.NET Web Application / Unit Test	Kovács Barnabás
Weboldal	Mucza János, Nagy Donát
Dokumentáció	Kovács Barnabás, Mucza János, Nagy Donát
Adatgyűjtés	Mucza János, Nagy Donát
Képek gyűjtése	Mucza János
Képekhez lekéréséhez használt szoftver beépítése	Mucza János

3. Adatbázis ER modell, leképezés



1. Ábra ER diagram

Tárolja a felhasználók adatait, foglalásukat, éttermeket, szállásokat, városokat és országokat. A rendszer célja, hogy nyomon kövesse a foglalási folyamatokat.

3.1. Táblák és mezők

Szállások (accommodation)

Mezők:

- accommodation_id(Primary key): szállás egyedi azonosítója
- name, address, rating: szállás egyéni adatai
- image: egy kép a szállásról
- city_id(Foreign Key): megmutatja, melyik városhoz tartozik

Országok (country)

Mezők:

- country_id(Primary key): ország egyedi azonosítója

- name: ország egyéni adata
- language: az adott országban beszélt nyelv

Városok (city)

Mezők:

- city_id(Primary key): város egyedi azonosítója
- name, zip_code: város egyéni adatai
- country_id(Foreign Key): megmutatja, melyik országhoz tartozik

Szobák (room)

Mezők:

- room_id(Primary key): szoba egyedi azonosítója
- room_number, capacity, price: szoba egyéni adatai
- accommodation_id(Foreign Key): megmutatja, melyik szálláshoz tartozik

Felhasználók (user)

Mezők:

- user_id(Primary key): felhasználó egyedi azonosítója
- name, phone: felhasználó egyéni adatai
- Password: titkosított vagy bináris formátumban tárolt jelszó (Token)
- is_admin: megmutatja, hogy admin-e a felhasználó
- token: titkosított szöveg, szükséges az engedélyek kiosztásához
- email(Unique key): felhasználó egyéni adata, csak egy lehet belőle

Szobafoglalások (roombooking)

Mezők:

- booking_id(Primary key): szobafoglalás egyedi azonosítója
- user_id, room_id, check_in, check_out(Foreign Key(s)): kapcsolók
- rating, created_at, status: szoba egyéni adatai

Éttermek (restaurant)

Mezők:

- restaurant_id(Primary key): étterem egyedi azonosítója
- name, address, rating: étterem egyéni adatai
- image: egy kép az étteremről
- city_id(Foreign Key): megmutatja, melyik városhoz tartozik

Éttermi asztalok (restauranttable)

Mezők:

- table_id(Primary key): éttermi asztal egyedi azonosítója
- seats: ülések száma
- restaurant_id(Foreign Key): megmutatja, melyik étteremhez tartozik

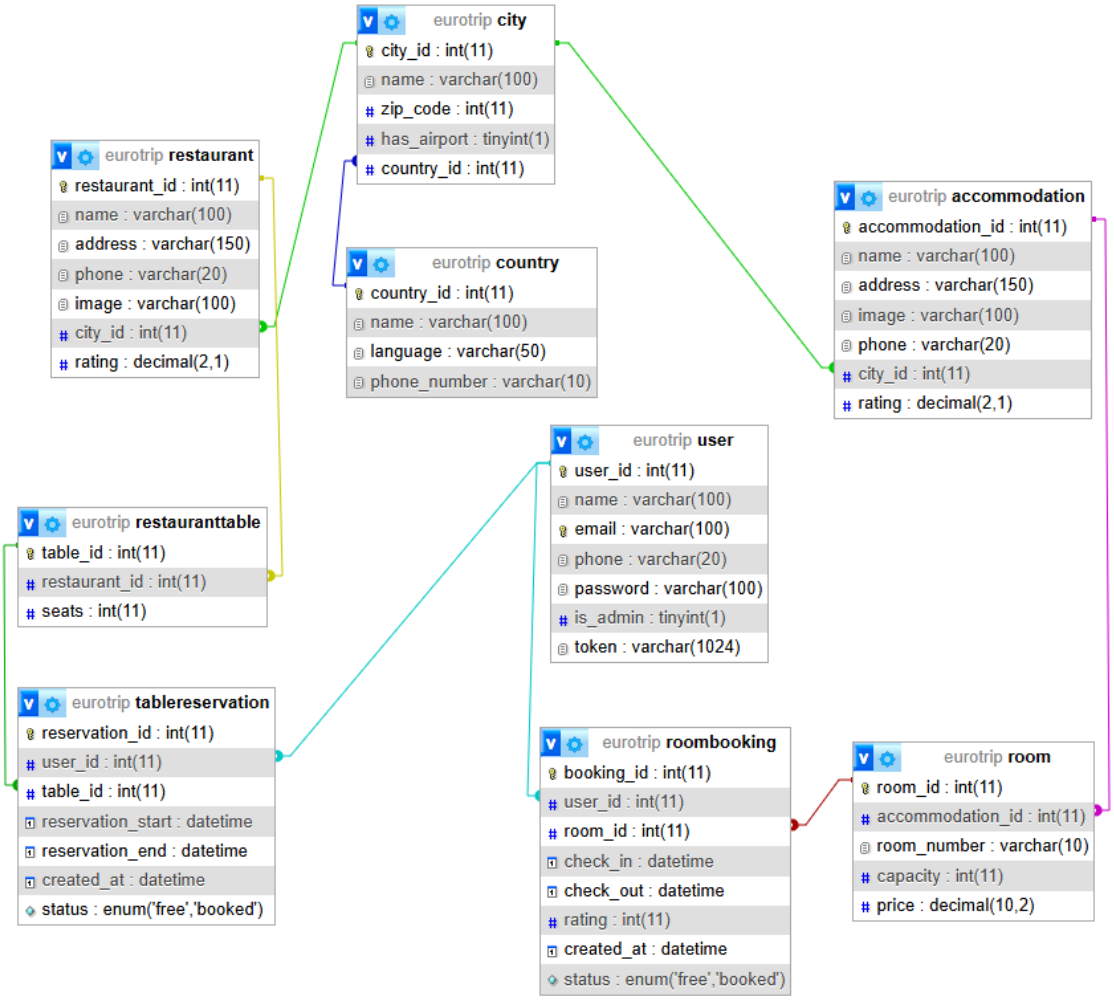
Asztal foglalások(tablereservation)

Mezők:

- reservation_id(Primary key): asztalfoglalás egyedi azonosítója
- user_id, table_id, reservation_start, reservation_end(Foreign Key(s)): kapcsolók
- created_at, status: asztalfoglalás egyéni adatai

Kapcsolatok (Foreign Keys)

- szállások - városok
- városok - országok
- szobák - szállások
- szobafoglalás - felhasználók, szobák
- éttermek - városok
- éttermi asztalok - éttermek
- asztalfoglalás - felhasználók, szobák



2. Ábra Adatbázis leképezés

4. REST API végpontok részletes specifikációja

A rendszer backendje ASP.NET Core technológiára épülő, REST API architektúrát használ. A kliens (React frontend) HTTP kéréseken keresztül kommunikál a szerverrel. Az adatok cseréje szabványos JSON formátumba történik, a pascalcase (C#) tulajdonságneveket a rendszer automatikusan camelCase formátumra alakítja a System.Text.Json beállítások révén. Az alábbiakban részletezzük a legfontosabb API kontrollereket, azok végpontjait, a szükséges jogosultságokat és a beépített üzleti logikát.

4.1. Hitelesítés és jogosultságkezelés (Login Controller)

Ez a kontroller felel a felhasználók beléptetéséért és a JWT (JSON Web Token) generálásáért.

- **POST** /api/login

- **Leírás:** Ellenőrzi a felhasználó hitelesítő adatait. A rendszer megkeresi az e-mail címet az adatbázisban, majd az egyedi PasswordHandler.VerifyPassword metódussal (Argon2id algoritmus) összeveti a jelszó hash-eket.
- **Jogosultság:** Bárki számára elérhető ([AllowAnonymus])
- **Kérés törzse (Body):** LoginRequestData objektum (UserId, Password).
- **Válasz (Siker):** 200 OK, a generált JWT tokennel (amely tartalmazza a jogosultsági Claim-eket).
- **Válasz (Hiba):** 401 Unauthorized, ha a jelszó vagy az e-mail hibás.

- **DELETE** /api/login/logout
  - **Leírás:** Kijelentkezteti a felhasználót. Mivel a JWT állapotmentes, ez a végpont a kliens oldali token törlését erősíti meg.
  - **Jogosultság:** Csak bejelentkezett felhasználók ([Authorzie]).

4.2. Városok kezelése és relációs adatlekérés (CityController)

A városok listázásáért és szűréséért felelős végpontok. Mivel a weboldal főoldalán a felhasználók városok alapján keresnek, ez az egyik leggyakrabban meghívott kontroller. A végpontok publikusak ([AllowAnonymous]), így bejelentkezés nélkül is böngészhető a kínálat.

- **GET** /api/City
  - **Leírás:** Lekéri az összes adatbázisban található várost.
  - **Üzleti és technikai logika:** A lekérdezés során az Entity Framework Core .Include(c=>c.Country) Linq metódusát alkalmaztuk. Ez az úgynevezett Eager Loading, magyarosítva mohó betöltés technika biztosítja azt, hogy a város adatai mellett a hozzátartozó ország adatai is azonnal, egyetlen SQL lekérdezésben betöltődjenek a memóriában. Így a frontend rögtön megkapja az ország nevét is, elkerülve az úgynevezett N+1 lekérdezési problémát, ami drasztikusan lassítaná a szervert.
  - **Válasz:** 200 OK, a városok és a beágyazott országok JSON listájával.
- **GET** /api/City/{id}
  - **Leírás:** Egy konkrét város adatainak lekérése az egyedi azonosítója alapján. Ezt a végpontot használja a CityDetails frontend komponens az oldal betöltésekor a város nevének és adatainak megjelenítéséhez.
  - **Válasz:** 200 OK, vagy 404 NotFound ha a város nem létezik.

- **GET** /api/City/country/{id}
    - **Leírás:** Városok szűrésére szolgál egy adott ország azonosítója (CountryId) szerint.
    - **Válasz:** 200 OK, ha van találat, különben 404 NotFound-ot kap vissza a felhasználó.

4.3. Országok listázása (CountryController)

Az országok lekérésre szolgáló kontroller ami szoros kapcsolatban van a városokkal az adatbázisban. A frontendben fontos szerepe van mivel a városok kártyáin meg van jelenítve.

- **GET /**api/Country
    - **Leírás:** Visszaadja a rendszerben elérhető összes országot és a hozzájuk tartozó adatokat. Ez a végpont elengedhetetlen a frontend lenyíló listáinak és a részletes keresőknek a feltöltéséhez.
    - **Válasz:** 200 OK, az országok listájával.
- **GET:** /api/Country/{id}
    - **Leírás:** Egy specifikus ország adatainak lekérése az azonosítója alapján.
    - **Válasz:** 200 OK, vagy hibás azonosító esetén 404 NotFound.

4.4. Szálláshelyek kezelése (AccommodationController)

- **GET** /api/Accomodation/city/{id}
    - **Leírás:** Lekéri egy adott városhoz (CityId) tartozó összes elérhető szálláshelyet. Ez a frontend város részletező oldalának (CityDetails) alapja.
    - **Jogosultság:** Szigorúan védett, csak adminisztrátorok számára
    - **Kérés törzse:** Accomodation objektum**.**
    - **Válasz:** 201 Created a létrehozott erőforrás elérhetőségével, vagy 400 BadRequest duplikáció esetében.
- **PUT** /api/Accomodation/{id} és **DELETE** /api/Accomodation/{id}
    - **Leírás:** Létező szállás adatainak módosítása vagy a szállás törlése az adatbázisból. A művelet előtt a rendszer validálja az ID létezését.
    - **Jogosultság:** Védett, az adminisztrátor kezeli.

4.5. Szobafoglalási rendszer (RoomBookingController)

A szoftver egyik legkritikusabb logikáját tartalmazó kontroller.

- **GET** /api/RoomBooking/me
    - **Leírás:** Lekéri az éppen bejelentkezett felhasználó saját foglalásait. A rendszer a tokenben lévő Email claim alapján azonosítja a felhasználót, majd az Entity Framework Include és ThenInclude metódusaival egy mély relációs lekérdezést végez, hogy a foglalás mellé csatolja a szoba, a szállás és a város adatait is.
    - **Jogosultság:** A bejelentkezett felhasználó (Policy="RoomBooking .ReadMe").
- **POST** /api/RoomBooking
    - **Leírás:** Új szobafoglalás leadása. A végpont egy összetett validációt hajt végre:
        1. Dátum validáció: A foglalás nem kezdődhet a múltban (CheckIn < DateTime.UtcNow.Date).
        2. Jövőbeli korlát: Maximum 6 hónapra előre lehet foglalni szállást.
        3. Ütközésvizsgálat (Overlap): Az Entity Framework Core segítségével kérdezi le, hogy van-e már aktív foglalás az adott RoomId-ra, ami időben átfedésben van a bevitt intervallummal.
- **Jogosultságkezelés:** Bejelentkezés szükséges, ennek hiányában nem lehet foglalást leadni.
- **Válasz (Hiba):** 409-es, Conflict típusú hibát jelez időpontüztközés esetén, 400-as, BadRequest-et hibás dátumoknál.
- **Válasz (Siker):** 201 Created, a foglalás automatikusan „booked"-ra változik, a CreatedAt mező pedig megkapja az aktuális időt, amit az API lekér.

4.6. Éttermek és asztalfoglalások (RestaurantController, TableReservationController)

Az éttermi egység a szállásokhoz hasonlóan épül fel, kiegészítve az asztalkapacitás kezeléssel.

- **GET /**api/Restaurant/city/{id}
    - **Leírás:** Egy város éttermeinek listázása szűrési célból. Amikor a weboldalon kiválasztunk egy várost, akkor a kiválasztott városon belül található étteremeket fogja kilistázni.
- **POST** /api/TableReservation
    - Leírás: Asztalfoglalás leadására szolgáló végpont.

- Logika: Hasonlóan a szobafoglaláshoz, itt is szigorú dátum/időellenőrzés fut. A rendszer megvizsgálja, hogy a kiválasztott asztal (TableId) az adott idősávban (ReservationStart - ReservationEnd) szabad-e. Átfedés esetén a foglalást a API 409 Conflict státuszkóddal visszautasítja.

4.7. Globális konfiguráció és biztonság (Program.cs)

Az API működésének alapjait a Program.cs fájlban konfiguráltuk:

- **CORS (Cross-Origin Resource Sharing):**
  - A CORS beállítás különösen lényeges modern webalkalmazások esetében, ahol a frontend és a backend gyakran külön környezetben fut. Az „AllowReact" pontosan meghatározza, hogy mely forrásból érkező kérések engedélyezettek, ezáltal csökkenti a jogosulatlan hozzáférések kockázatát, miközben zavartalan kommunikációt biztosít a React alapú kliens és az API között.
- **JSON Ciklusok megakadályozása:**
  - A JSON szerializáció kezelése szintén kulcsfontosságú. Az Entity Framework által kezelt összetett objektumkapcsolatok könnyen vezethetnek körkörös hivatkozásokhoz, amelyek nemcsak hibát okozhatnak, hanem feleslegesen megnövelhetik a válasz méretét is. A ReferenceHandler.IgnoreCycles alkalmazásával ezek a problémák elegánsan kiküszöbölhetők, így a kliensoldali feldolgozás gyorsabbá és megbízhatóbbá válik.

Összességében ezek a beállítások hozzájárulnak egy jól strukturált, biztonságos és hatékony API kialakításához, amely képes kiszolgálni a modern frontend alkalmazások igényeit anélkül, hogy kompromisszumot kellene kötni a teljesítmény vagy a megbízhatóság terén.

5. Biztonsági architektúra és hitelesítés

Az EuroTrip alkalmazás védelme érdekében egy többszintű biztonsági rendszert implementáltunk, amely a legmodernebb kriptográfiai megoldásokra és állapotmentes (stateless) hitelesítésre épül.

5.1. Kriptográfiai jelszóvédelem (Argon2id)

A rendszerben a jelszavak soha nem tárolódnak nyers szövegként. A PasswordHandler osztály az **Argon2id** algoritmust használja, amely jelenleg a legbiztonságosabb választás a brute-force támadások ellen.

- **Só (Salt) alkalmazása:** Minden regisztrációkor egy egyedi 16 bájtos kriptográfiai véletlen értéket (salt) generálunk a RandomNumberGenerator segítségével.
- **Számítási paraméterek:** Az algoritmus 4 iterációval és 65.536 KB (64 MB) memóriahasználattal dolgozik, ami megnehezíti a hardveres (GPU/ASIC) feltörési kísérleteket.
- **Tárolási formátum:** Az adatbázisban a generált sót és a hash értéket egyetlen sztringként, ponttal (.) elválasztva tároljuk a user táblában.
- **Hitelesítés:** Bejelentkezéskor a rendszer nem dekódolja a jelszót (mivel a hash egyirányú), hanem a megadott jelszóból és a tárolt sóból újraszámítja a hash-t, majd CryptographicOperations.FixedTimeEquals metódussal veti össze őket az időzített támadások elkerülése érdekében.

```
public static class PasswordHandler
{
    public static string HashPassword(string password)
    {
        // Salt generálása
        byte[] salt = RandomNumberGenerator.GetBytes(16);

        var argon2 = new Argon2id(Encoding.UTF8.GetBytes(password))
        {
            Salt = salt,
            DegreeOfParallelism = Environment.ProcessorCount,
            Iterations = 4,
            MemorySize = 65536
        };

        byte[] hash = argon2.GetBytes(32);

        // Salt + hash Base64-be kódolva
        return $"{Convert.ToBase64String(salt)}.{Convert.ToBase64String(hash)}";
    }
```

3. Ábra Jelszó hashelés

5.2. JWT alapú hitelesítés és token menedzsment

Mivel az API RESTful architektúrát követ, a hitelesítés állapottartás nélkül, **JSON Web Token (JWT)** használatával történik

- **A Token felépítése:** A sikeres bejelentkezés után a szerver egy digitálisan aláírt tokent állít ki. Ez tartalmazza a felhasználó e-mail címét és a hozzá rendelt engedélyeket.

- **Digitális aláírás:** A tokeneket egy 256 bites titkos kulccsal írjuk alá **HMAC SHA256** algoritmussal, garantálva, hogy a kliensoldalon az adatok ne legyenek módosíthatóak!
- **Lejárati idő:** A biztonság fokozása érdekében a tokenek érvényességi idejét 1 órában maximalizáltuk.

```
public string GenerateToken(User user)
{
    var claims = new List<Claim>
    {
        new (ClaimTypes.Name, user.Email)
    };

    foreach (var permission in _rolesPermissions[user.isAdmin!])
    {
        claims.Add(new Claim("permission", permission));
    }

    var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_secretKey));
    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

    var token = new JwtSecurityToken(
        issuer: _issuer,
        audience: _audience,
        claims: claims,
        expires: DateTime.Now.AddHours(1),
        signingCredentials: creds
    );

    return new JwtSecurityTokenHandler().WriteToken(token);
}
```

4. Ábra Token generálás

5.3. Szerepkör alapú jogosultságkezelés

Az alkalmazás különbséget tesz az adminisztrátorok és a regisztrált felhasználók között a User modell isAdmin mezője alapján.

- **Dinamikus engedélyek:** A TokenManager a konfigurációból tölti be, hogy melyik szinthez milyen engedélyek (pl.: AccomodationCreate, RoomBooking.ReadMe) tartoznak.
- **API védelem:** A kontrollerek végpontjait specifikus politikákkal (Policy) védjük a [Authorize] attribútum segítségével. Ez biztosítja, hogy például egy szálláshelyet csak az arra jogosult adminisztrátor törölhessen

```
"IsAdmin": {
  "1": [
    "Accommodation.Create",
    "Accommodation.Read",
    "Accommodation.ReadId",
    "Accommodation.ReadCityId",
    "Accommodation.Update",
    "Accommodation.Delete",
    "City.Read",
    "City.ReadId",
    "City.ReadCountryId",
    "Country.Read",
    "Country.ReadId",
    "Login.Create",
    "Login.Delete",
    "Restaurant.Create",
    "Restaurant.Read",
    "Restaurant.ReadId",
    "Restaurant.ReadCityId",
    "Restaurant.Update",
    "Restaurant.Delete",
    "RestaurantTable.Create",
    "RestaurantTable.ReadId",
    "RestaurantTable.Update",
    "RestaurantTable.Delete",
    "RoomBooking.Create",
    "RoomBooking.ReadMe",
    "RoomBooking.Read",
    "RoomBooking.ReadId",
```

5. Ábra Felhasználói szerepkörök egy része

Tábla / Entitás	***Engedélyezett műveletek (CRUD)***	***Jogosultsági szint (Szerepkör)***	***Üzleti indoklás és logikai háttér***
Szállás (Accommodation)	C, R, U, D	Olvasás: Felhasználó CRUD: Adminisztrátor	A publikus adatokat minden látogató olvashatja. A rendszer adatminőségének megőrzése érdekében új szálláshelyeket kizárólag az adminisztrátorok hozhatnak létre, módosíthatnak vagy törölhetnek a megfelelő JWT token birtokában.
Ország (Country)	R (Csak olvasás)	Olvasás: Felhasználó	Az országok listája statikus szótáradat, amely a rendszer indulásakor kerül inicializálásra. A weboldalon keresztül történő módosításukra nincs üzleti igény, így csak lekérdező (GET) végpontok készültek hozzájuk.
Város (City)	R (Csak olvasás)	Olvasás: Felhasználó	A városok adatai elengedhetetlenek a főoldali kereső és a szűrőfunkciók működéséhez. Ezek az adatok ritkán változnak, adminisztrációjuk közvetlenül az adatbázis-kezelőn keresztül történik.
Szoba (Room)	C, R, U, D	Olvasás: Felhasználó CRUD: Adminisztrátor	A szobák listázása a szálláshelyek adatlapján történik. Az adminisztrátorok jogosultak új szobákat (kapacitás, ár) hozzárendelni egy meglévő szállodához a kapacitásbővítés lekövetése érdekében.

Felhasználó (User)	C, R, U, D	Létrehozás: Felhasználó (Regisztráció) Olvasás, Frissítés: Bejelentkezett felhasználó CRUD: Adminisztrátor	Bárki létrehozhat magának fiókot (Create). A bejelentkezett felhasználók a profil oldalon megtekinthetik (Read) és frissíthetik (Update) a saját adataikat.
Szobafoglalás (RoomBooking)	C, R, U, D	Létrehozás, Olvasás, Törlés(Sajátját):Felhasználó CRUD: Adminisztrátor	A regisztrált felhasználók rögzíthetik foglalásaikat (Create) az időpont-ütközési validációk sikeres lefutása után, és lekérdezhetik a saját történetüket (Read: /api/RoomBooking/me).
Étterem (Restaurant)	C, R, U, D	Olvasás: Felhasználó CRUD: Adminisztrátor	A vendéglátóhelyek böngészése publikus. Az adatbázis karbantartását, új éttermek felvételét és a képek/címek frissítését kizárólag a rendszergazdák végezhetik.
Éttermi Asztal (RestaurantTable)	C, R, U, D	Olvasás: Felhasználó CRUD: Adminisztrátor	Az éttermek férőhely-kapacitását az asztalok határozzák meg. Ezek módosítására (pl. selejtezés, új asztal beszerzése) csak adminisztrátori jogosultsággal van lehetőség.
Asztalfoglalások (TableReservation)	C, R, U, D	Létrehozás, Olvasás, Törlés(Sajátját):Felhasználó CRUD: Adminisztrátor	A felhasználók lefoglalhatják a szabad asztalokat (Create). A rendszer biztosítja, hogy senki ne férjen hozzá mások foglalási adataihoz, csak a sajátjához.

5.4. A regisztrációs folyamat logikája

A felhasználók létrehozása a backend oldalon egy többlépcsős validációs és titkosítási folyamaton megy keresztül:

1. **Adatok validálása:** A rendszer ellenőrzi hogy minden kötelezően kitöltendő mező ki van-e töltve. Kötelezővé a mezőket a required kulcsszóval tudjuk tenni.
2. **Egyediség ellenőrzése:** Az adatbázis lekérdezésével meggyőződünk arról, hogy az adott e-mail címmel vagy névvel nem regisztráltak-e már korábban.
3. **Biztonságos hash-elés:** Ha a validáció sikeres, a PasswordHandler.HashPassword metódus legenerálja a jelszó titkosított változatát.
4. **Perzisztencia:** A rendszer elmenti a felhasználót az adatbázisba, de a jelszót kizárólag a {Base_Salt}.{Base64_Hash} formátumba rögzíti. Ezzel biztosítjuk, hogy még az adatbázis esetleges kiszivárgása esetén sem fejthetőek vissza a felhasználók eredeti jelszavai.

```
[AllowAnonymous]
[HttpPost("register")]
public async Task<IActionResult> PostRegister([FromBody] User user)
{
    if (string.IsNullOrWhiteSpace(user.Name) ||
        string.IsNullOrWhiteSpace(user.Password) ||
        string.IsNullOrWhiteSpace(user.Email)) return BadRequest("Hiányzó adatok!");

    // 1. Ellenőrizzük, létezik-e már a felhasználónév VAGY az email
    var existingUser = await _context.Users
        .FirstOrDefaultAsync(u => u.Name == user.Name || u.Email == user.Email);

    if (existingUser != null)
    {
        // Pontosabb hibaüzenet (opcionális)
        if (existingUser.Email == user.Email)
            return BadRequest("Ez az email cím már regisztrálva van!");

        return BadRequest("Ez a felhasználónév már foglalt!");
    }
```

6. Ábra Regisztráció folyamata

```
    // 2. Jelszó hashelése és mentés
    user.Password = PasswordHandler.HashPassword(user.Password);
    user.isAdmin = 0; // Biztonság kedvéért kényszerítsük sima felhasználónak

    _context.Users.Add(user);
    await _context.SaveChangesAsync();

    return Created($"{Request.GetDisplayUrl()}/{user.Id}", user);
}
```

7. Ábra Jelszó titkosítás és mentés

5.5. Hitelesítés és jelszóellenőrzés

Bejelentkezéskor a rendszer a LoginController segítségével azonosítja a felhasználót:

- A megadott e-mail alapján lekéri a tárolt hash-t az adatbázisból.
- A PasswordHandler.VerifyPassword metódus segítségével, a tárolt passwordsalt felhasználásával újra kiszámítja a hash-t a megadott jelszóból, és összehasonlítja a tárolttal.
- A biztonság fokozása érdekében a rendszer "konstans idejű" összehasonlítást alkalmaz, hogy elkerülje az időzítés alapú (timing attack) támadásokat.

```
public static bool VerifyPassword(string password, string storedHash)
{
    var parts = storedHash.Split('.');
    byte[] salt = Convert.FromBase64String(parts[0]);
    byte[] stored = Convert.FromBase64String(parts[1]);

    var argon2 = new Argon2id(Encoding.UTF8.GetBytes(password))
    {
        Salt = salt,
        DegreeOfParallelism = Environment.ProcessorCount,
        Iterations = 4,
        MemorySize = 65536
    };

    byte[] computed = argon2.GetBytes(32);

    return CryptographicOperations.FixedTimeEquals(stored, computed);
}
```

8. Ábra Hitelesítés

6. Kliensoldali architektúra és felhasználói felület (Frontend)

Miután biztonságos háttérrendszer (ASP.NET Core API) és az adatbázis megtervezésre került, a következő kritikus lépés az adatok vizuális prezentálása és a felhasználói interakciók biztosítása volt. A frontend fejlesztéséhez a legmodernebb technológiákat választottuk, hogy a végeredmény egy gyors, reszponzív és felhasználóbarát webalkalmazás legyen.

6.1. Reaktív komponens alapú felépítés

Az alkalmazás logikai és vizuális egységeit kisméretű, újrafelhasználható komponensekre bontottuk. Ez a megközelítés lehetővé teszi a kód tisztán tartását és a párhuzamos fejlesztést.

- **Virtual DOM és hatékonyság:** A React Virtual DOM technológiáját használjuk, amely garantálja, hogy csak azok a felületi elemek frissüljenek, amelyekben tényleges adatváltozás történt. Például Dashboard.jsx komponensben a városok szűrésekor a böngésző nem tölti újra a teljes oldalt, csupán a kártyák listáját rendereli újra a memóriában számított különbségek alapján.
- **Deklaratív UI:** A felület állapotát (State) a React hook-ok (useState, useEffect) kezelik. A CityDetails.jsx fájlban látható, hogyan váltunk a "Szálláshelyek" és "Éttermek" fülek között: egy egyszerű activeTab állapot változása azonnal kiváltja a megfelelő komponens (AccomodationDetails.jsx vagy RestaurantDetails.jsx)megjelenítését.

6.2. Globális konfiguráció és belépési pont(main.jsx)

Az alkalmazás inicializálása a `main.jsx` fájlban történik. Itt konfiguráltuk a rendszer működéséhez elengedhetetlen "szolgáltatókat" (Providers):

- **QueryClientProvider:** A TanStack query motorját teszi elérhetővé az egész alkalmazás számára, kezelve az API kérések gyorsítótárazását.
- **BrowserRouter:** Biztosítja a HTML5 history API alapú navigációt, lehetővé téve a tiszta URL-ek használatát.

6.3. Dinamikus útvonalválasztás (React Router)

Az App.jsx komponens felelős az alkalmazás útvonal felépítésének definiálásáért. A react-router-dom könyvtár segítségével dinamikus paraméterezést alkalmazunk:

- **Dinamikus útvonalak:** A <Route path="/citys/:cityId" element={<CityDetails />} /> sor mutatja meg a rendszer rugalmasságát. A :cityId egy változó, amelyet a CityDetails komponens a useParams hook segítségével olvas ki, hogy lekérje a specifikus város adatait a backendről.
- **Átirányítások és fallback:** A rendszer tartalmaz egy catch-all útvonalat (path="*"), amely minden nem létező URL esetén visszairányítja a felhasználót a kezdőlapra, elkerülve a holtpontokat a navigációban.

6.4. Aszinkron állapotkezelés

Az Eurotrip egyik legfejlettebb technikai megoldása az aszinkron szerverállapot kezelése. Ahelyett, hogy manuális fetch hívásokat és komplex useEffect láncokat használnunk mindenhol, a **TanStack query (React Query)** könyvtárat integráltuk.

- **Caching kezelése:** A useCurrentUser.js hookban konfigurált staleTime: 5 * 60 * 1000 paraméter biztosítja, hogy a bejelentkezett felhasználó adatait 5 percig a memóriából olvassa ki a rendszer, radikálisan csökkentve a redundáns hálózati kérések számát.
- **Automatikus betöltési állapotok:** Minden lekérdező hook (pl. useRoomsByAcco) automatikusan biztosít egy isLoading változót. Ezt a UI komponensekben (pl. AccommodationDetails.jsx) arra használjuk, hogy egy egységes betöltő animációt (loading-spinner) jelenítsünk meg, amíg az adatok meg nem érkeznek.

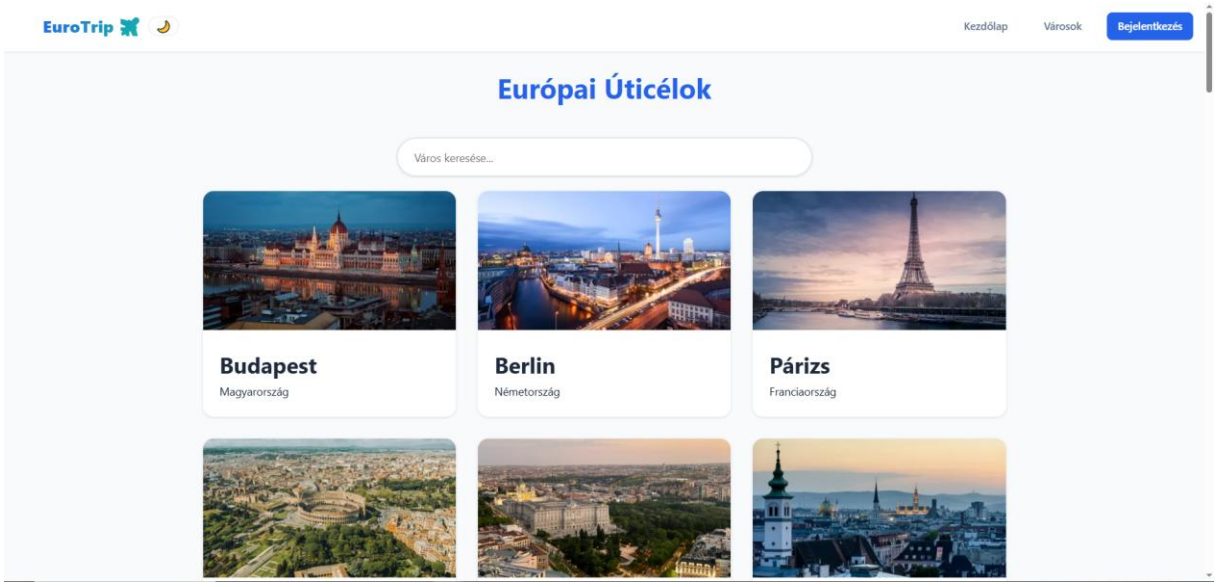
6.5. A felhasználói felület részletes komponens-elemzése

Az EuroTrip frontendje több tucat kisebb egységből áll, amelyek közül a legfontosabbak felelősek az üzleti logika és a vizuális megjelenítés összekapcsolásáért. Az alábbiakban részletezzük a rendszer gerincét alkotó komponenseket.

6.5.1. Dashboard: A központi kereső és városkezelő:

A Dashboard komponens az alkalmazás "szíve", amely a városok listázásáért és a szűrésért felel. Itt jelenik meg az összes város

- **Keresési logika**: A komponens egy searchCity állapotot használ a felhasználói bevitel követésére. A listázás nem közvetlenül a szerverről érkező tömbön, hanem egy filteredCities segédváltozón fut végig, amely valós időben veti össze a városneveket a keresési kulcsszóval.
- **Kép-URL generálási algoritmus:** A getCityImgUrl függvény egy komplex transzformációt hajt végre: a városneveket normalizálja (ékezetmentesítés), kisbetűssé alakítja, majd a szóközöket kötőjelekre cseréli, hogy megfeleljenek a Cloudinary tárolási konvencióinak. Ez biztosítja, hogy például a "Bécs" nevű város automatikusan a becs.jpg-t hívja meg.

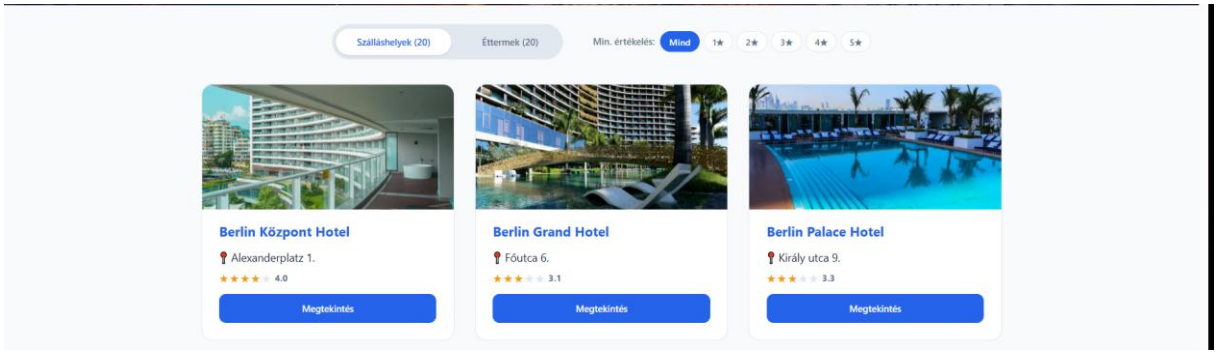


9. Ábra Dashboard

6.5.2. CityDetails: Dinamikus adataggregáció

Ez a komponens felelős egy adott város teljes turisztikai kínálatának (szállások és éttermek) összefogásáért..

- **Fülek közötti váltás:** Az activeTab állapot segítségével a felhasználó zökkenőmentesen válthat a szálláshelyek és éttermek listája között. A React feltételes renderelése (conditional rendering) biztosítja, hogy csak a kiválasztott kategória elemei kerüljenek a DOM-ba.
- **Értékelési szűrő:** Bevezettünk egy minRating állapotot, amely lehetővé teszi a találatok további finomítását. A rendszer csak azokat a kártyákat jeleníti meg, amelyek értékelése eléri vagy meghaladja a felhasználó által kiválasztott csillagszámot.
- **Alkomponensek használata:** A kód tisztasága érdekében az értékeléseket a StarRating, a képeket pedig a DynamicImage segédkomponens kezeli, utóbbi beépített betöltő animációval (loading spinner) rendelkezik.



10. Ábra Szállások és éttermek közötti váltás

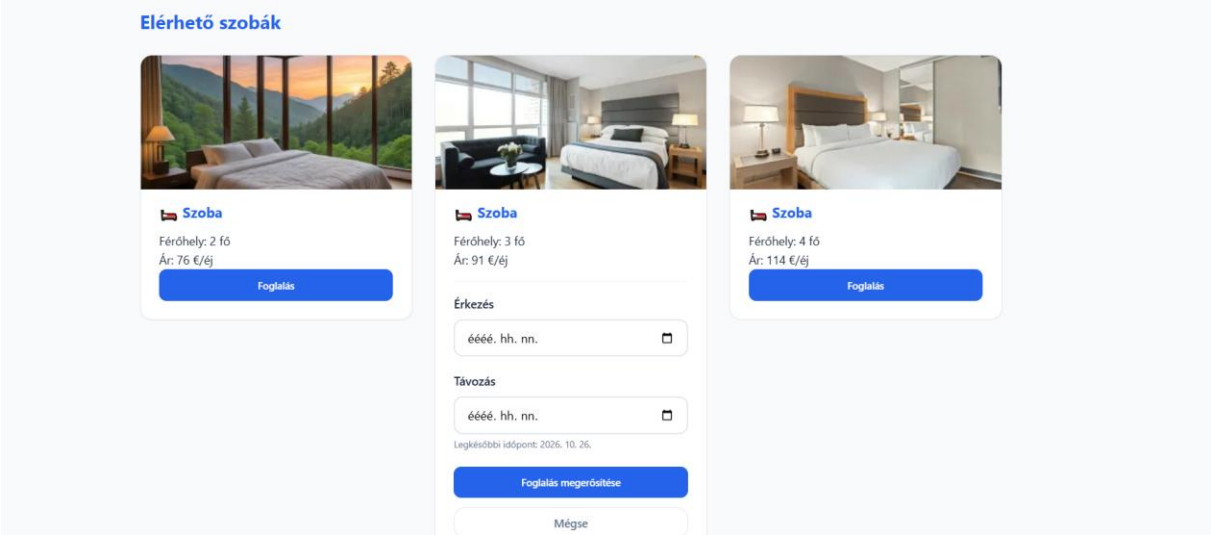
6.5.3. AccommodationDetails és RestaurantDetails:

Ezek a komponensek kezelik a tényleges foglalási folyamatokat, így itt kapott helyet a legösszetettebb űrlap-logika. Az egyik legfontosabb része ezeknek a komponenseknek, hogy csak bejelentkezett állapotban lehet megtekinteni a szobákat/asztalokat illetve lefoglalni őket. A legfontosabb működési alapelvek:

- **Űrlap állapotkezelés:** A foglalási adatokat (dátumok, létszám) egy belső form objektum tárolja.
- **Szigorú dátum-validáció:** A kliensoldalon dinamikusan számítjuk ki a today (mai nap) és a maxDate (ma + 6 hónap) értékeket. Ezeket a HTML input mezők min és max

attribútumaiként adjuk át, így a böngésző natívan akadályozza meg a múltbéli vagy túl távoli foglalásokat.

- **Aszinkronitás:** A foglalás gomb a useBookRoom mutációs hook isPending állapotától függően válik inaktívvá, megelőzve a hálózati késés miatti többszörös beküldést.



11. Ábra Szoba/asztal foglalása

6.5.4. Profil: Személyre szabott felhasználói központ

A Profile komponens egy integrált felület a felhasználói adatok módosítására és a korábbi foglalások kezelésére.

- **Múltbeli adatok listázása:** A komponens párhuzamosan hívja meg a useMyBookings és useMyReservations hook-okat.
- **Állapotjelzők (Status Badges):** A foglalások aktuális státuszát (pl. "booked") a StatusBadge komponens alakítja vizuálisan értelmezhető formába, CSS osztályok segítségével színezve az elemeket.
- **Interaktív törlés:** A foglalások lemondása a deleteBooking és deleteReservation funkciókon keresztül történik, amelyek egy megerősítő ablak (window.confirm) után hajtják végre az API hívást, majd frissítik a gyorsítótárat.

6.5.5. Szigorú validáció regisztráláskor (Register.jsx):

A **szigorú validáció** lényege, hogy a felhasználó már a frontend oldalon visszajelzést kap arról, hogy az általa megadott adatok megfelelőek-e. Ez nemcsak a

felhasználói élményt javítja, hanem csökkenti a backend terhelését is, hiszen hibás adatokat eleve nem engedünk tovább.

- Egy komplex isFormValid logikai változót hoztunk létre, amely folyamatosan figyeli, hogy az e-mail tartalmaz-e '@' és '.' karaktereket, a jelszó eléri-e a minimum 6 karaktert, és a "Jelszó megerősítése" mező pontosan megegyezik-e az eredeti jelszóval.
- A "Regisztráció" gomb mindaddig szürkített (opacity: 0.6) és inaktív marad, amíg az összes feltétel nem teljesül.

7. Dizájn és CSS

A projekt stíluskezelésének alapköve a változó-alapú megközelítés. Az App.css fájl elején, a :root szelektorban definiáltuk a globális design-tokeneket.

- **Színpaletta:** Meghatároztunk egy elsődleges kék színt (--primary: #2563eb) a márkajelzéshez, egy türkiz kiemelő színt (--accent) a cselekvésre ösztönző gombokhoz, valamint specifikus szemantikus színeket a hibákhoz (--error) és a sikeres műveletekhez (--success).
- **Vizuális megjelenés:** Egységesítettük a lekerekítéseket (--radius: 16px) és az árnyékokat (--shadow-md), ami professzionális, modern hatást kölcsönöz a kártyáknak és az űrlapoknak.
- **Tipográfia:** A rendszer az 'Inter' és 'Segoe UI' betűcsaládokat preferálja, amelyek kiváló olvashatóságot biztosítanak minden kijelzőtípuson.

```
:root {
    --primary: #2563eb;
    --primary-hover: #1d4ed8;
    --accent: #14b8a6;
    --accent-hover: #0f766e;
    --bg: #f8fafc;
    --card-bg: #ffffff;
    --text-main: #1e293b;
    --text-muted: #64748b;
    --white: #ffffff;
    --error: #ef4444;
    --success: #22c55e;
    --shadow-sm: 0 1px 3px rgba(0,0,0,0.1);
    --shadow-md: 0 4px 6px -1px rgba(0,0,0,0.1);
    --shadow-lg: 0 10px 15px -3px rgba(0,0,0,0.1);
    --radius: 16px;
}
```

12. Ábra App.css

7.1. Layout: flexbox és grid kapcsolata

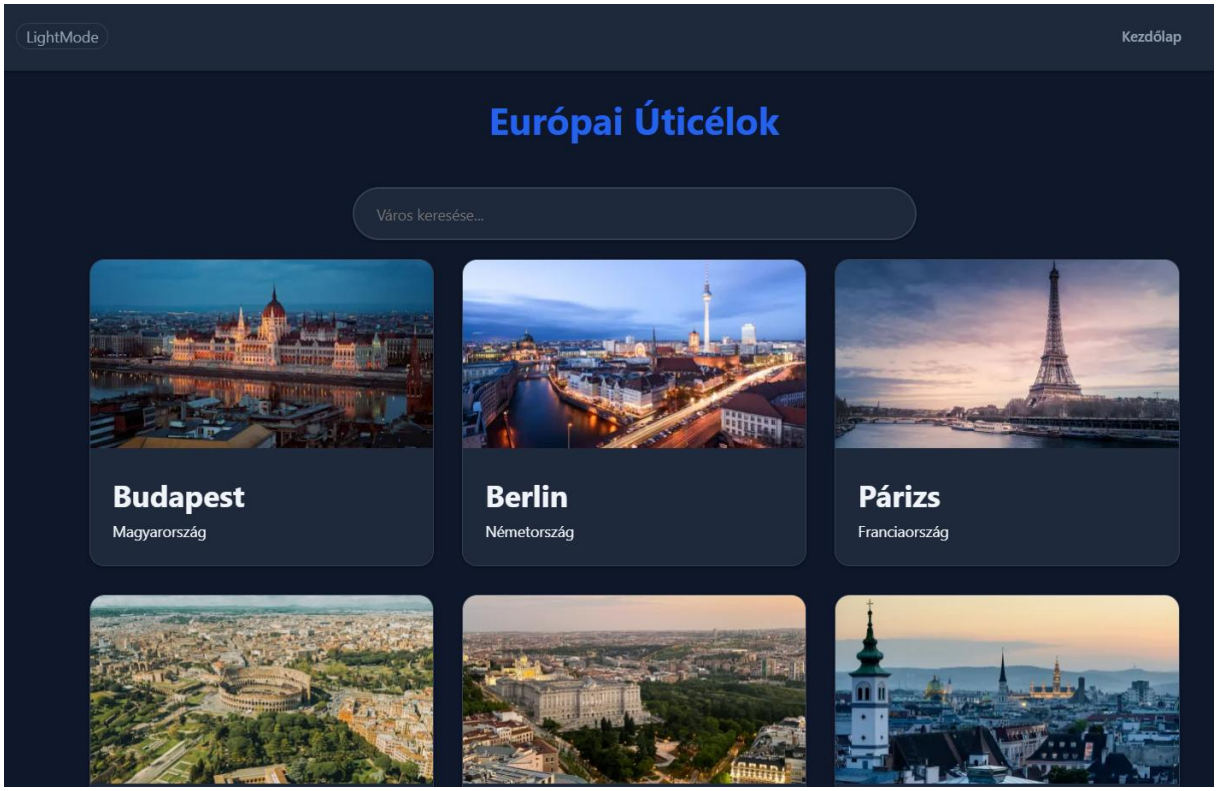
A felület elrendezéséhez kizárólag modern CSS elrendezési modelleket használtunk, elkerülve a régi, "float"-alapú megoldásokat.

- **Dinamikus rácsháló (CSS grid):** A városok és szállások listázásához a grid rendszert alkalmaztuk. A grid-template-columns: repeat(auto-fill, minmax(300px, 1fr)) szabály automatikusan kezeli a kártyák számát: ha van hely, több oszlop jelenik meg, ha a kijelző szűkül, a kártyák rugalmasan egymás alá rendeződnek anélkül, hogy manuális töréspontokat kellene megadnunk.
- **Rugalmas igazítás (flexbox):** A navigációs sáv (navbar), a lábléc (footer) és a kártyák belső tartalma Flexbox segítségével igazodik. Ez teszi lehetővé például a logó és a menüpontok széthúzását (justify-content: space-between), vagy a gombok függőleges középre igazítását a kártyák alján.

7.2. Dinamikus témakezelés és sötét mód

Az EuroTrip egyik kiemelt funkciója a beépített sötét mód, amely kíméli a felhasználó szemét gyenge fényviszonyok között.

- **Implementáció:** A témaváltás nem külön stíluslapokkal, hanem a CSS változók felülírásával történik. Amikor az alkalmazás a [data-theme="dark"] attribútumot kapja, a háttérszín (--bg), a kártyaszín (--card-bg) és a szövegszínek változói automatikusan frissülnek a sötétebb tónusokra.
- **UX előny:** Ez a technika biztosítja a folyamatos átmenetet; a felhasználónak nem kell az oldal újratöltésére várnia, a változás azonnal és villódzásmentesen történik meg.



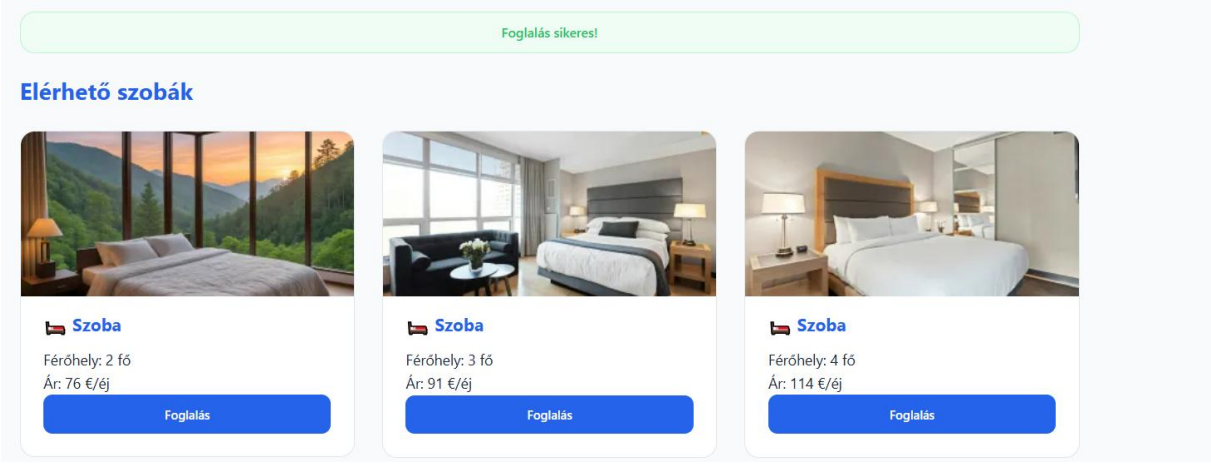
13. Ábra Sötét mód

7.3. Vizuális megjelenés és kártyás felépítés

A turisztikai információk tálalásához egyedi "Sub-card" dizájnt hoztunk létre.

- **Negatív margó technika:** A kártyákon belül a képek (.image-wrapper) negatív margót kaptak (margin: -25px -25px 16px -25px), hogy kitöltsék a kártya teljes szélességét a felső élig, modern, magazin-szerű megjelenést biztosítva.
- **Interaktivitás:** A kártyákra lebegtetéskor (hover) egy finom elmozdulás (transform: translateY(-8px)) és egy erősebb árnyék

kerül, ami egyértelmű visszajelzést ad a felhasználónak a kattinthatóságról.

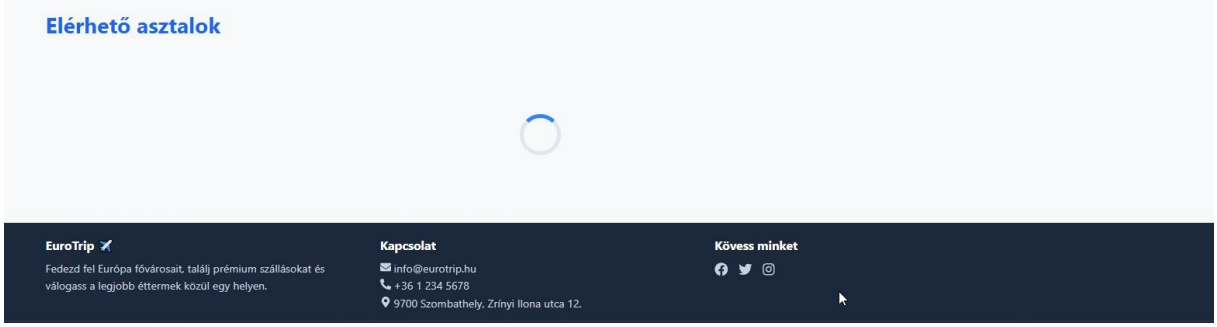


14. Ábra Rendszer visszajelzés

7.4. Animációk és felhasználói visszajelzések

A felhasználó élményt finom animációkkal fokoztuk.

- **Beúszó effektusok:** A kártyák és az űrlapok a fadeIn animáció segítségével jelennek meg, ami egy 20 pixeles függőleges mozgást és az átlátszóság fokozatos növelését foglalja magában.
- **Betöltési visszajelzés:** Mivel az adatlekérés aszinkron, egy egyedi spin animációval ellátott töltőikont (spinner) alkalmazunk. Ez csökkenti a felhasználó bizonytalanságát a várakozási idő alatt.



15. Ábra Betöltési animáció

7.5. Reszponzivitás

Az alkalmazás "Mobile-First" szemléletben készült, biztosítva a tökéletes megjelenést okostelefonon is.

- **Média lekérdezések:** A 768px alatti szélességű kijelzőkre specifikus szabályokat határoztunk meg: a címsorok betűmérete csökken, a navigációs sáv függőleges elrendezést vesz fel, és az űrlapok kitöltik a teljes képernyőszélességet a kényelmesebb gépelés érdekében.

8. Weboldal technikai leírása

8.1. Komponenseinek technikai leírása

1. Frontend: Reaktív UI komponens-alapú architektúra

A frontend nem csupán egy weboldal, hanem egy **állapotvezérelt alkalmazás (Stateful Application)**.

- **Virtual DOM & Reconciliation:** A React minden változásnál (pl. város szűrése) egy virtuális másolatot készít a felületről, összehasonlítja a valósággal (diffing algoritmus), és csak a legszükségesebb pixelmódosításokat hajtja végre. Ez biztosítja a villámgyors navigációt.
- **Aszinkron Lifecycle Management:** A useEffect hookok segítségével kezeljük az API-hívások életciklusát. Ez biztosítja, hogy a szobák adatai csak akkor töltődjenek be, amikor a felhasználó ténylegesen a CityDetails komponensre lép, elkerülve a felesleges hálózati forgalmat.
- **Dynamic Route Matching:** A react-router-dom segítségével a :cityId paraméter alapján dinamikusan generálunk tartalmat. Ez teszi lehetővé, hogy egyetlen komponens (CityDetails) szolgáljon ki több ezer különböző várost.
- **Media Optimization:** A Cloudinary-val való integráció során nem fix képeket töltünk be, hanem **URL-transzformációkat** hajtunk végre. A w_1200,h_400,c_fill paraméterekkel kényszerítjük a szervert, hogy a képet már a felhőben vágja és tömörítse (WebP formátumba), mielőtt az a böngészőbe érne.

2. Backend: ASP.NET Core REST API & Middleware Pipeline

A backend egy **Dependency Injection (DI)** konténerre épülő rendszer, ahol minden osztály (Controller, Context) lazán kapcsolt.

- **Request Pipeline & Middleware:** Minden kérés átmegy egy sor "szűrőn" (Middleware). Itt történik a **CORS-politika** ellenőrzése, a **JWT hitelesítés** dekódolása és a hibakezelés (Exception Handling), még mielőtt a kérés elérné a kontrollert.

- **Controller-Action Pattern:** Az ApiController attribútummal ellátott osztályok végzik az útválasztást. A C# aszinkron (async/await) programozási modelljét használjuk, ami lehetővé teszi, hogy a szerver ne blokkolja a szálat, amíg az adatbázis válaszára vár (non-blocking I/O).
- **Identity & Security:** A jelszavakat soha nem tároljuk plain textben. A rendszer a **BCrypt** vagy hasonló algoritmust használva "sózott" hash-eket hoz létre. A belépés után generált JWT token tartalmazza a felhasználó Claim-jeit (pl. Role=Admin), amit a szerver minden későbbi kérésnél validál a titkos kulccsal.

3. Adatelérés: Entity Framework Core & SQL Relációs Modell

Az adatok kezelése nem manuális SQL parancsokkal, hanem egy **ORM (Object-Relational Mapping)** rétegen keresztül történik.

- **LINQ (Language Integrated Query):** A C# kódban írt lekérdezéseket az EF Core fordítja le optimalizált SQL parancsokká. Példa: A _context.Rooms.Where(r => r.AccommodationId == id) kódból a háttérben egy SELECT * FROM room WHERE accommodation_id = @id lesz.
- **Database First & Schema Integrity:** Az eurotrip.sql alapján látható, hogy szigorú **idegen kulcs (Foreign Key)** megkötéseket alkalmazunk. Ez garantálja, hogy ne lehessen szobát foglalni egy nem létező szállodába (Referenciális integritás).
- **Connection Pooling:** A MySQL-hez való kapcsolódás során a rendszer "kapcsolati medencét" használ, így nem kell minden egyes kéréshez újra felépíteni a TCP/IP kapcsolatot, ami jelentősen gyorsítja az adatlekérést.

4. Hálózati protokollok és adatátvitel

- **Stateless Protocol:** Az API teljesen állapotmentes. Minden kérésnek tartalmaznia kell minden szükséges információt (például az Authorization headerben a tokent), így a szerver bármikor skálázható.
- **JSON Serialization:** A System.Text.Json könyvtár felelős azért, hogy a komplex C# objektum-gráfokat (pl. Város -> Szállások -> Szobák) körkörös hivatkozásmentes JSON formátummá alakítsa a frontend számára.

8.2. Működésének műszaki feltételei

A működtetéshez szükséges backend és adatbázis kapcsolat, amely a háttérből küldi az erőforrásokat a frontend komponensnek. Az adatok áramlásakor biztosítani kell az adatok titkosítását (felhasználó adatai), amelyhez külső szolgáltatást használunk: JSON WebToken.

8.3. Használatának rövid bemutatása

A felhasználó ellátogat az oldalra, amely egy kezdőlappal fogadja. Onnan hivatkozások segítségével tud navigálódni. Ezek a hivatkozások ugyanúgy a városokhoz vezetik, melyeket szűrni is tudja egy kereső sáv segítségével. Egy adott város kiválasztását követően láthatja az éttermeket, szállásokat, majd a szabad asztalokat-szobákat. A foglaláshoz szükséges egy fiókot regisztrálni, ha ez már megtörtént, akkor csak a bejelentkezés szükséges.

- **Adatbázis**

  Szükséges egy MySQL szerver telepítése, majd az adatbázis-dump beimportálása.

  A MySQL szerver a 3306-os porton fut, a backend is.

  Példa user: -email: pelda@gmail.com, jelszó: pelda123

- **Frontend**

  A frontend futtatásához Node.v24.14.1-es verzió szükséges npm-mel. További frontendhez szükséges csomagok:

  -Tanstack Query, globális állapotmenedzselő könyvtár

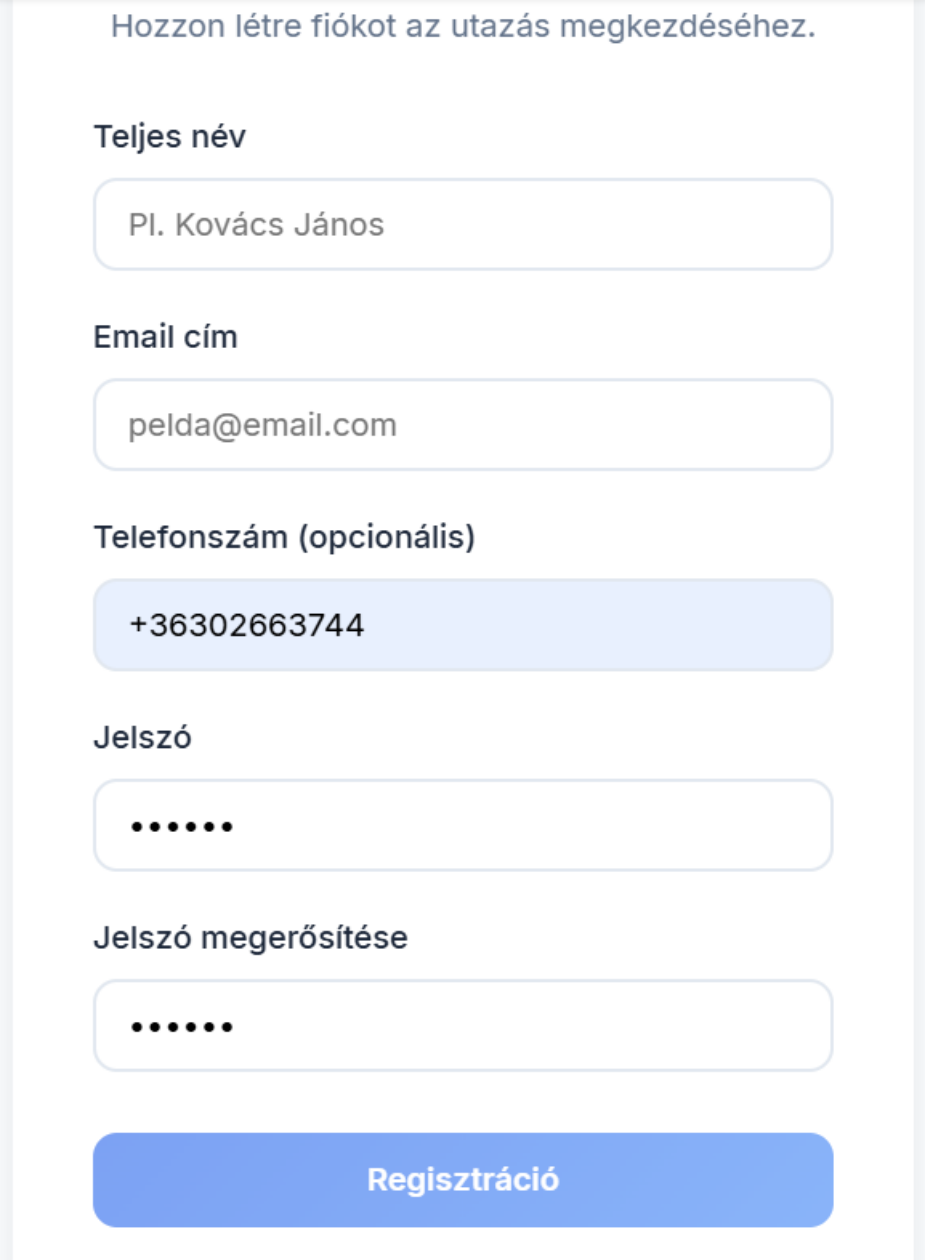
  -React Router

  Miután a csomagok sikeresen települtek, npm run dev parancs kiadása szükséges. A frontend alapvetően https-en és localhost-on próbál kapcsolatba lépni a backend-el.

- **Backend**

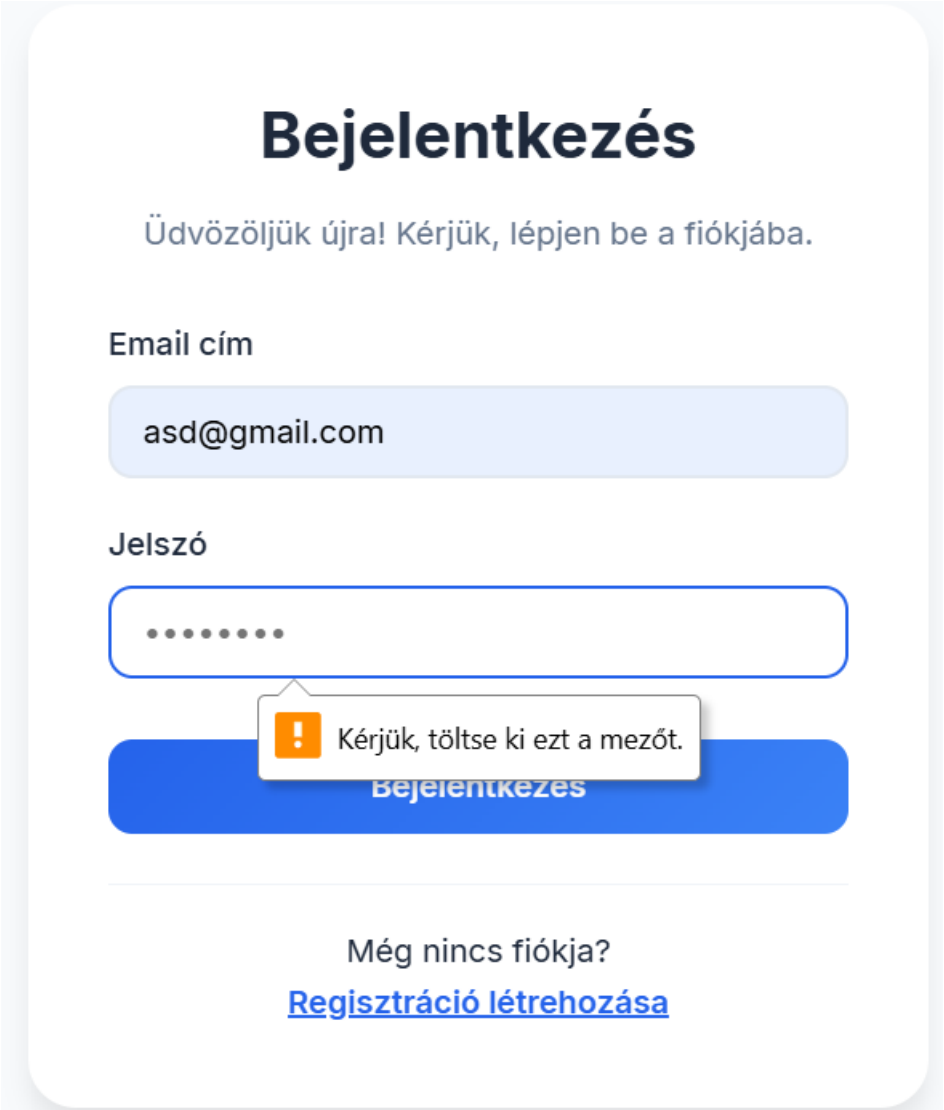
  A szoftver backend része ASP.NET Core 8-as verzión alapul és C#-ban íródott. Visual Studio-ból futtatható. A projekt építése (build) során ez elméletileg letölti a szükséges NuGet csomagokat, mint a MySQL.EntityFrameworkCore, a Moq. A projekt https-en a 7046-os porton fut, http-n pedig az 5177-esen.

Regisztráció



16. Ábra Regisztráció

Bejelentkezés



17. Ábra Bejelentkezés

9. Tesztek

A szoftver minőségének biztosítása miatt az automatizált tesztelés mellett döntöttünk, amely garantálja, hogy a kód módosítása során nem romlik el a már meglévő háttér logika. Az EuroTrip API háttérrendszerének ellenőrzésére a Microsoft által fejlesztett **MSTest** keretrendszert alkalmaztuk.

9.1. Tesztelési környezet

A tesztek izolált, biztonságos futtatása érdekében elkerültük a fejlesztői MySQL adatbázis használatát. Ehelyett egy memóriában futó **SQLite In-Memory** adatbázist építettünk be.

- **Előnye:** A tesztek villámgyorsan lefutnak, mivel nincsenek lemezműveletek. A tesztek nem szemetelik tele az éles adatbázist, és minden tesztfutás tiszta, alap állapotból indul.
- **Párhuzamosítás:** Az MSTestSettings.cs fájlban beállítottuk az [assembly: Parallelize(Scope = ExecutionScope.MethodLevel)] attribútumot, amely engedélyezi a tesztmetódusok párhuzamos futtatását, így maximalizálva a processzormagok kihasználtságát és minimalizálva a tesztelés idejét.

9.2. Adatbázis implementálása

A DbContextHelper osztály felel a tesztkörnyezet felépítéséért.

- A CreateDbContext() metódus minden teszt inicializálásakor megnyit egy új SQLite kapcsolatot, létrehozza a sémát (EnsureCreated()), majd feltölti az adatbázist előre definiált, statikus tesztadatokkal (Dummy Data).
- **Párhuzamosítás:** A tesztadatok (például UserList, AccommodationList, CityList) garantálják, hogy a tesztesetek mindig kiszámítható adathalmazon dolgozzanak, így egy lekérdezés eredménye determinisztikus marad.

9.3. Objektum összehasonlítás

A tesztek írása során fellépő gyakori probléma, hogy a memóriában lévő (várt) objektumok és az adatbázisból visszakapott objektumok memóriacíme eltér, így a hagyományos egyenlőségvizsgálat (Assert.AreEqual) hamis hibát adna. Ennek kiküszöbölésére minden entitáshoz saját összehasonlító Comparer osztályt írtunk, amelyek az IEqualityComparer<T> interfészt valósítják meg.

- Például az AccommodationComparer a Equals metódusban felülbírálja az alapértelmezett logikát, és a tulajdonságok (Id, Name, Address, Image, CityId) értékalapú egyezését vizsgálja. Ez garantálja a pontos és robusztus teszt-asszerciókat (assertions).

```
6   namespace EuroTripTest
7   {
8       internal class DbContextHelper
9       {
10          public List<User> UserList = [
11                  new User { Id = 1, Name = "Albert", Email = "a", Phone="06", Password="al", isAdmin=0},
12                  new User { Id = 2, Name = "Bertha", Email = "b", Phone="06", Password="be", isAdmin=0 },
13                  new User { Id = 3, Name = "Ceasar", Email = "c", Phone="06", Password="ce", isAdmin=0 }
14              ];
15          public List<Accommodation> AccommodationList = [
16                  new Accommodation { Id = 1, Name = "AlbertHotel", Address = "a", Image="a.png", Phone="06", CityId=1 },
17                  new Accommodation { Id = 2, Name = "BellaHotel", Address = "b", Image="b.png", Phone="06", CityId=2 },
18                  new Accommodation { Id = 3, Name = "CeasarHotel", Address = "c", Image="c.png", Phone="06", CityId=3 }
19              ];
20          public List<City> CityList = [
21                  new City { Id = 1, Name = "Ajka", HasAirport=1, ZipCode=10234, CountryId=1 },
22                  new City { Id = 2, Name = "Buda", HasAirport=1, ZipCode=10235, CountryId=2 },
23                  new City { Id = 3, Name = "Celdolk", HasAirport=0, ZipCode=10236, CountryId=3}
24              ];
25          public List<Country> CountryList = [
26                  new Country { Id = 1, Name = "Hungary", PhoneNumber="0630", Lang="Hungarian" },
27                  new Country { Id = 2, Name = "Hungary", PhoneNumber = "0630", Lang = "Hungarian" },
28                  new Country { Id = 3, Name = "Ciprus", PhoneNumber = "0730", Lang = "Turkish"}
29              ];
30          public List<Restaurant> RestaurantList = [
31                  new Restaurant{ Id = 1, Name = "AjkaRest",  Address = "a", Image="a.png", Phone="06", CityId=1 },
32                  new Restaurant{ Id = 2, Name = "BudaRest",  Address = "b", Image="b.png", Phone="06", CityId=2 },
33                  new Restaurant{ Id = 3, Name = "CeldolkRest",  Address = "c", Image="c.png", Phone="06", CityId=3 },
34              ];
35          public List<RestaurantTable> RestaurantTableList = [
36                  new RestaurantTable { Id = 1, RestaurantId=1 },
37                  new RestaurantTable { Id = 2, RestaurantId=2 },
38                  new RestaurantTable { Id = 3, RestaurantId=3 }
39              ];
40          public List<RoomBooking> RoomBookingList = [
41                  new RoomBooking { Id = 1, UserId = 1, RoomId=1, CheckIn=new DateTime(2026,2,28,5,0,0), CheckOut=new DateTime(2026,3,10,5,0,0), Rating=
```

18. Ábra DbContextHelper.cs

9.4. Tesztosztályok

Minden API kontrollerhez külön tesztosztályt hoztunk létre (pl. AccommodationControllerTest, CityControllerTest). Minden tesztosztály a következő életciklust követi:

1. **Setup**: Példányosítja a DbContextHelper-t, és létrehozza a tesztelendő objektumot (SUT - System Under Test), injektálva a memória-adatbázist. Továbbá egy DefaultHttpContext-et is beállít, hogy szimulálja a webes HTTP kéréseket.
2. **Kikérdezés:** Meghívja a kontroller aszinkron metódusait.
3. **Ellenőrzés:** Validálja az eredményt.

```
namespace EuroTripTest
{
    internal class UserComparer : IEqualityComparer<User>
    {
        public bool Equals(User? x, User? y)
        {
            return x == null && y == null ||
                x?.Id == y?.Id &&
                x?.Name == y?.Name &&
                x?.Email == y?.Email &&
                x?.Phone == y?.Phone &&
                x?.Password == y?.Password &&
                x?.isAdmin == y?.isAdmin;
        }

        public int GetHashCode([DisallowNull] User obj)
        {
            return obj.Id;
        }
    }
}
```

19. Ábra Összehasonlítás

9.5. Részletes CRUD tesztelés

Az alapvető lekérdezések mellett minden entitásnál leteszteltük a teljes életciklust egyetlen összetett tesztmetódusban (pl. TestAll_Accommodation). Ez a folyamat a következőképpen néz ki:

- **POST:** Létrehozunk egy új szállást. Ellenőrizzük, hogy a válasz CreatedResult-e, és hogy a rendszer visszaadja-e a generált egyedi azonosítót.
- **GET:** A frissen kapott Id alapján lekérjük az adatot, és érvényesítjük, hogy a név (Test Hotel) megegyezik-e a várt értékkel.
- **PUT:** Átírjuk a memóriában lévő objektum nevét, elküldjük a PUT kérést, majd egy újabb GET lekérdezéssel bizonyítjuk, hogy az adatbázisban is megtörtént a frissítés.
- **DELETE:** Kiadjuk a törlési parancsot, majd megpróbáljuk újra lekérni az elemet. Az Assert.IsNull igazolja, hogy az elem véglegesen eltűnt a rendszerből.

10. Segédoldalak, információs felületek, linkek:

Programunk megvalósításához több helyről is vettünk segítségeket, képeket ezzel gyorsítva, megkönnyítve a folyamatokat.

Ezek között volt:

- Az órán megtanult tudás alkalmazása.
- Kódok újra hasznosítása.
- Cloudinary felhőalapú képtároló
- Mesterséges Intelligencia (stílus, logo tervezés)

11. Továbbfejlesztési lehetőségek

Az EuroTrip webalkalmazás jelenlegi formájában is egy teljesen működőképes és a fő célokat ellátó szoftver, a fejlesztés során rengeteg új ötletünk támadt, amiket idő hiányában a későbbi verziókba tervezünk beépíteni. A legfontosabb reális továbbfejlesztési irányok a következők:

- **Valós online fizetési rendszer:** Jelenleg az alkalmazásban a szobákat és asztalokat "csak" lefoglalni lehet, a fizetés a helyszínen történik. Egy nagyon hasznos lépés lenne egy külső fizetési szolgáltató (például a Stripe) beépítése, így a felhasználók azonnal, bankkártyával is kifizethetnék a szállásukat.
- **Szöveges értékelések és kommentek**: Most a szállások és éttermek csak egy 1-től 5-ig terjedő csillagos értékeléssel rendelkeznek. Sokkal jobb és hasznosabb lenne, ha a felhasználók a látogatás után egy szöveges véleményt is tudnának írni a szállás oldalára, segítve ezzel a többi utazó döntését.
- **Automatikus e-mail értesítések:** Jó lenne, ha a rendszer nemcsak a weboldalon belül, a profil menüben mutatná a sikeres foglalást, hanem a háttérben küldene egy automatikus visszaigazoló e-mailt is a felhasználóknak.
- **Térkép beépítése:** Bár minden szállásnak és étteremnek meg van adva a pontos címe az adatbázisban, sokat dobna a felhasználói élményen, ha a kártyák alatt megjelenne egy Google Térkép is. Így a látogatók rögtön látnák, hogy hol helyezkedik el a hotel/étterem.
- **Különálló adminisztrációs grafikus felület:** A backend kódunk már fel van készítve az adminisztrátori jogosultságokra, de jelenleg nincs egy kifejezetten nekik szóló, zárt webes felület. Egy "Admin Dashboard" kialakításával a weboldal tulajdonosai informatikai tudás nélkül, egyszerű űrlapokon keresztül tudnának új városokat, képeket vagy szállásokat feltölteni a rendszerbe.

- **Nyelvválasztás:** Mivel egy európai utazásokat célzó oldalról van szó, logikus lenne a weboldal angol, illetve német nyelvre történő lefordítása, hogy a nemzetközi látogatókat is kiszolgálhassa.
- **Környezetvédelem**: Végezetül, a hosszú távú fenntarthatóság jegyében a platformba beépíthető egy „Zöld lábnyom" mérő is, amely az utazások szén-dioxid-kibocsátását kalkulálná, és környezetbarát alternatívákat – például vonatutakat vagy öko-tanúsítvánnyal rendelkező szállásokat – kínálna fel kiemelt helyen, ezzel megszólítva a környezettudatos modern utazók egyre növekvő rétegét.

Ábrajegyzék

1. Ábra ER diagram 6
2. Ábra Adatbázis leképezés 9
3. Ábra Jelszó hashelés 14
4. Ábra Token generálás 15
5. Ábra Felhasználói szerepkörök egy része 16
6. Ábra Regisztráció folyamata 18
7. Ábra Jelszó titkosítás és mentés 18
8. Ábra Hitelesítés 19
9. Ábra Dashboard 21
10. Ábra Szállások és éttermek közötti váltás 22
11. Ábra Szoba/asztal foglalása 23
12. Ábra App.css 25
13. Ábra Sötét mód 26
14. Ábra Rendszer visszajelzés 27
15. Ábra Betöltési animáció 27
16. Ábra Regisztráció 31
17. Ábra Bejelentkezés 32
18. Ábra DbContextHelper.cs 34
19. Ábra Összehasonlítás 35